

Parameter-Efficient Adaptation of TrOCR for Devanagari Handwritten Word Recognition

Samir Wagle¹ and Manish ...¹

Tribhuvan University, Kathmandu, Nepal
`samir@redtab.xyz`

Abstract. Devanagari handwritten text recognition lags Latin-script systems, and the transformer models that close this gap are costly to fine-tune for low-resource settings. We present a systematic study of parameter-efficient adaptation of TrOCR for Devanagari handwritten word recognition on the IIIT-INDIC-HW-WORDS (Hindi) benchmark. We ablate LoRA rank and target-module placement against full fine-tuning and zero-shot bounds, and show that adapters training 2.5% of model parameters recover 93% of the full fine-tuning improvement (CER 0.1695 $\rightarrow$ 0.1013 vs. 0.0961), with feed-forward placement beating higher attention rank at a matched budget. We further identify input-distribution parity as a dominant and previously unreported factor: fine-tuning on raw word strips while the base checkpoint expects foreground-cropped, square-padded inputs *degrades* recognition below the unadapted baseline (CER 0.5297 $\rightarrow$ 0.1013 after alignment), an effect invisible to training loss. Finally, we introduce akshara-level evaluation for Devanagari: an orthographic-syllable error rate and an error taxonomy separating conjunct, matra, and nasalization-sign failures, showing that codepoint-level CER systematically misrepresents which linguistic units fail. Code, adapters, and evaluation tooling are released.

Keywords: Handwritten text recognition · Devanagari · TrOCR · LoRA · Parameter-efficient fine-tuning.

1 Introduction

Handwritten text recognition (HTR) for Devanagari — the script of Hindi, Nepali, Marathi, and Sanskrit, used by over 600 million people — remains substantially behind Latin-script HTR. The script’s properties make word recognition harder than its Latin counterpart: consonants combine with dependent vowel signs (*matras*) above, below, before, and after the base glyph; consonant clusters fuse into *conjunct* ligatures (e.g. क्ष, त्र) whose shapes are not compositional; and the headline (*shirorekha*) connects glyphs within a word, defeating segmentation-based pipelines.

Transformer encoder-decoder models such as TrOCR [5] transfer well to new scripts, but full fine-tuning of a $\sim$ 190M-parameter model is expensive in the low-resource settings where Indic HTR is most needed. Low-rank adaptation

(LoRA) [4] promises the same adaptation at a fraction of the trainable parameters, but no systematic study exists for Devanagari: which layers to adapt, at what rank, and what the gap to full fine-tuning is.

This paper contributes:

1. **A systematic PEFT study for Devanagari word recognition.** We ablate LoRA rank ($r \in \{4, 8, 16, 32\}$) and target-module placement (attention-only vs. attention+FFN) on IIIT-INDIC-HW-WORDS Hindi [3], against zero-shot and full fine-tuning bounds.
2. **An input-parity finding.** Fine-tuning on raw word images while the base checkpoint was trained on foreground-cropped, square-padded inputs *reduces* word accuracy from 0.1192 to ~ 0.02 despite converging training loss — the model visibly loses matra recognition. Restoring preprocessing parity flips the result. We argue input-distribution parity is a necessary control that PEFT studies on pretrained OCR models currently omit.
3. **Akshara-level evaluation.** We propose evaluating Devanagari HTR at the orthographic-syllable (akshara) level, with an error taxonomy that separates conjunct substitutions, matra errors, and nasalization-sign errors. This reveals failure modes that codepoint CER conflates.

2 Related Work

Transformer HTR and TrOCR. TrOCR [5] pairs a ViT encoder with an autoregressive text decoder, pretrained on synthetic and labeled text lines. It transfers to new scripts by replacing the tokenizer and fine-tuning; community checkpoints exist for Devanagari [8], and our study adapts one such checkpoint rather than the English original, isolating the adaptation question from the script-transfer question.

Parameter-efficient fine-tuning for OCR. DLoRA-TrOCR [2] applies weight-decomposed LoRA to TrOCR for mixed handwritten/printed English text, establishing that PEFT is viable for OCR transformers. Our work differs in three ways: we study an abugida whose orthography stresses the decoder differently (matras, conjuncts); we ablate *where* adapters go, not only their rank; and we evaluate at the akshara level. To our knowledge no PEFT study exists for any Indic script.

Indic HTR. IIIT-INDIC-HW-WORDS [3] established word-level benchmarks across ten Indic scripts with CRNN-style baselines. Subsequent Devanagari work has largely used CNN/RNN hybrids; transformer adaptation results on this benchmark are scarce, and none report parameter-efficiency trade-offs.

Synthetic handwriting for augmentation. Latent diffusion generators such as WordStylist [6] and DiffusionPen [7] produce training data that improves Latin-script HTR. No such generator exists for Devanagari; we develop one, and the corresponding augmentation study, in a companion paper.

3 Method

3.1 Base Model and LoRA Placement

We adapt `trocr-devanagari-2` [8], a VisionEncoderDecoder checkpoint (ViT-base encoder, byte-level BPE decoder tokenizer covering Devanagari) previously fine-tuned for Devanagari, totaling 184M parameters. LoRA adapters with scaling $\alpha = 2r$ and dropout 0.05 are inserted into one of three module sets:

- **attn**: encoder/decoder attention projections (query, key, value, output);
- **attn+ffn**: additionally the feed-forward projections;
- **legacy**: the suffix-matched set `{query, key, value, dense}` used by our originally deployed adapter, which via suffix matching also captures encoder FFN layers — included to quantify the effect of unintentional placement.

At $r=16$ these train 2.36M (attn) to 4.62M (legacy) parameters — under 3% of the model. Full fine-tuning (184.08M) provides the upper bound; the unadapted checkpoint provides the lower.

3.2 Input-Distribution Parity

The base checkpoint was trained on word images that were foreground-cropped and, for aspect ratios above 2.2, padded to a white square before ViT resizing. The benchmark’s raw images are wide strips (typical AR > 3). Passing them directly to the ViT processor squashes them anisotropically — a different input distribution from the checkpoint’s.

We therefore train and evaluate under *preprocessing parity*: the same crop/pad transform as the checkpoint’s original regime. Section 5.2 quantifies what happens without it. We treat this as an experimental control, not a trick: any PEFT study on a pretrained OCR model implicitly chooses an input distribution, and the choice can dominate the effect being measured.

3.3 Akshara-Level Evaluation

Codepoint-level CER treats the conjunct ऋ (three codepoints) and the simple consonant क (one) asymmetrically, and scatters a single perceptual error across multiple codepoint edits. We segment NFC-normalized text into *aksharas* — orthographic syllables of the form $(C \text{ virama})^* C (\text{nukta})? (\text{matra})? (\text{sign})^*$ — and report:

- **AER**: Levenshtein distance over akshara sequences, normalized by reference length;
- **an error taxonomy** from the akshara-level alignment: *conjunct substitution* (either side contains a virama-joined cluster), *matra error* (same consonant skeleton, different vowel marking), *nasalization-sign error* (anusvara/candrabindu/visarga only), *base substitution*, and insertions/deletions.

All text is Unicode-NFC-normalized before scoring; without this, visually identical predictions differing only in codepoint composition inflate CER.

Table 1. Test-split results on IIIT-INDIC-HW-WORDS-Hindi. CER: character error rate (NFC codepoints); AER: akshara error rate; WAcc: word recognition accuracy.

Run	Trainable	CER ↓	AER ↓	WAcc ↑
full_ft	184.08M	0.0961	0.1192	0.7980
lora_r16_legacy	4.62M	0.1013	0.1307	0.7786
lora_r16_attn_ffn	4.62M	0.1022	0.1315	0.7774
lora_r32_attn	4.72M	0.1030	0.1326	0.7760
lora_r16_attn	2.36M	0.1065	0.1368	0.7691
lora_r8_attn	1.18M	0.1095	0.1411	0.7614
lora_r4_attn	0.59M	0.1141	0.1466	0.7534
base_prep	184.08M	0.1695	0.2213	0.6461

4 Experimental Setup

Data. IIIT-INDIC-HW-WORDS Hindi [3, 1]: $\sim 70\text{k}$ train / 12.9k validation / 12.9k test word images from the official splits. No writer overlap handling is needed at word level. **Training.** Seed 42; AdamW; LR 5×10^{-5} , batch 16, 2 epochs (LoRA) / LR 2×10^{-5} , batch 8, 1 epoch (full FT); beam search (4 beams) for generation; max generation length 32 tokens. All runs fit a single NVIDIA T4 (Kaggle free tier) at ~ 70 – 90 minutes per LoRA configuration, making the study reproducible without datacenter hardware. **Metrics.** CER with 95% percentile-bootstrap CIs (1,000 resamples), AER (Sect. 3.3), and word recognition accuracy, all on the official test split.

5 Results

5.1 Main Comparison and LoRA Ablation

Table 1 reports the full matrix; Fig. 1 plots CER against trainable parameters.

Rank. With attention-only placement, CER falls monotonically with rank — 0.1141 ($r=4$) $\rightarrow$ 0.1095 ($r=8$) $\rightarrow$ 0.1065 ($r=16$) $\rightarrow$ 0.1030 ($r=32$) — but with diminishing returns: each rank doubling buys roughly 0.4 CER points, then 0.3, then 0.35. **Placement.** At a matched parameter budget ($\sim 4.6\text{M}$), moving capacity into feed-forward modules beats raising attention rank: attn+ffn at $r=16$ (0.1022) and the legacy suffix-matched set (0.1013) both outperform attention-only $r=32$ (0.1030). **Gap to full fine-tuning.** The best adapter (legacy, 4.62M trainable parameters — 2.5% of the model) reaches CER 0.1013 against 0.0961 for full fine-tuning of all 184.08M parameters. A paired bootstrap over the 12,869 test words puts the difference at +0.41 CER points, 95% CI [0.08, 0.76] ($p \approx 0.009$, 2,000 resamples): full fine-tuning is statistically better, but the adapter recovers 93% of the zero-shot $\rightarrow$ full-FT improvement (CER 0.1695 $\rightarrow$ 0.0961) — and lifts word accuracy from 0.6461 to 0.7786 — while training $40\times$ fewer parameters. **Seed robustness.** Repeating both headline configurations across three seeds (42–44) gives CER 0.1020 ± 0.0010 (best adapter) and

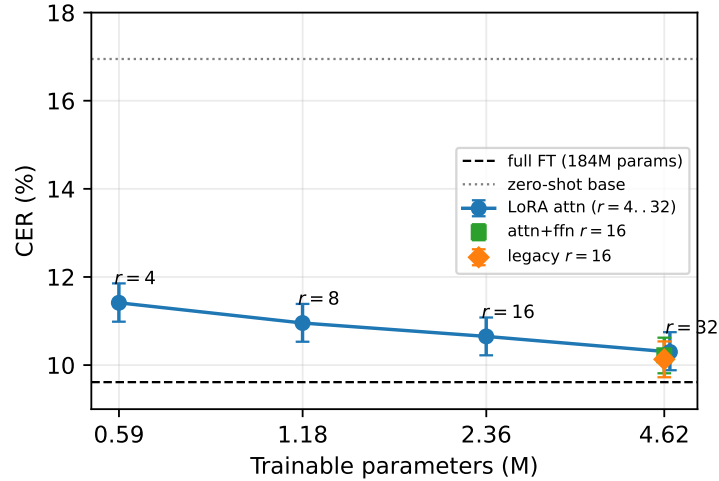


Fig. 1. Test CER (%; 95% bootstrap CIs) vs. trainable parameters. The attention-only rank curve improves monotonically but sub-linearly in $\log r$; at a matched ~ 4.6 M-parameter budget, spending capacity on FFN modules (attn+ffn, legacy) outperforms doubling attention rank ($r=32$).

0.0953 ± 0.0008 (full FT; mean $\pm$ std): seed variance is an order of magnitude smaller than every effect we report.

5.2 The Cost of Ignoring Input Parity

Under the raw-input regime, one epoch of LoRA fine-tuning converges in training loss ($6.25 \rightarrow 0.34$) yet *collapses* generation: word accuracy falls from 0.1192 (unadapted, raw inputs) to 0.022, with predictions retaining consonant skeletons but dropping matras (अनार्थो $\rightarrow$ अनथथय). With preprocessing parity restored, the same 400-sample smoke configuration reaches CER 0.178 — and the unadapted baseline itself improves from 0.5297 to 0.1695. Training loss is computed under teacher forcing and cannot reveal this failure; only generation-based evaluation does. We recommend PEFT studies on pretrained OCR models report their input pipeline as part of the method.

5.3 Akshara-Level Error Analysis

Table 2 breaks the akshara-level alignment into script-aware categories. Two findings stand out. First, **conjuncts are the hard case**: aksharas containing virama-joined clusters err at rate 0.170 (full FT) and 0.190 (LoRA) against 0.108 and 0.115 for simple aksharas — roughly $1.6\times$ — and words with 2+ conjuncts carry nearly twice the akshara error rate of conjunct-free words (0.20 vs. 0.11).

Table 2. Akshara-level error taxonomy on the test set (42,374 reference aksharas), for full fine-tuning and the best adapter.

Error category	full FT		LoRA legacy $r=16$	
	Count	%	Count	%
Base consonant substitution	2015	39.1	2220	40.1
Conjunct substitution	1608	31.2	1580	28.6
Deletion	530	10.3	583	10.5
Matra (dependent vowel)	512	9.9	586	10.6
Insertion	230	4.5	280	5.1
Nasalization/visarga sign	174	3.4	197	3.6
Conjunct insertion/deletion	83	1.6	85	1.5
Total akshara edits	5152		5531	

Image	Reference	Predictions (full FT / LoRA)
	प्रोसैसर	both correct
	पदोन्नति	both correct
	अत्यधिक	अर्थशक / अर्थशक (conjunct)
	ह्रास	द्वास / द्वास (conjunct)
	अपनाता	अपनाना / अपनाना (base subst.)
	दुश्मन	दुश्मन / दुःशम (LoRA only)

Fig. 2. Qualitative examples. Conjunct-dense words (प्रोसैसर, पदोन्नति) are often recognized exactly, but cursive or ambiguous conjuncts (त्य, ह्र) trigger substitutions that both models share; the last row shows a failure unique to the adapter.

Second, **script-specific categories account for $\sim 45\%$ of all errors**: conjunct substitutions ($\sim 30\%$), matra errors ($\sim 10\%$), and nasalization-sign errors ($\sim 3.5\%$) have no Latin-script analogue and are invisible to plain CER, which scatters a single conjunct confusion across up to three codepoint edits. The taxonomy also shows *where* the extra capacity of full fine-tuning goes: relative to LoRA it removes matra errors (-13%) and base substitutions (-9%), while conjunct substitutions are essentially unchanged ($+2\%$), suggesting the residual conjunct difficulty is a visual-encoder limitation that neither adapter capacity nor full fine-tuning closes. Fig. 2 shows representative cases.

6 Conclusion

We presented the first systematic parameter-efficient fine-tuning study for Devanagari handwritten word recognition, an input-parity control that any adaptation study on pretrained OCR checkpoints should adopt, and akshara-level

metrics that expose script-specific failure modes invisible to codepoint CER. Adapters training 2.5% of parameters recover 93% of the full fine-tuning gain, making high-quality Devanagari HTR trainable on free-tier hardware. The error taxonomy identifies conjunct recognition as the open problem: its error rate is $1.6\times$ that of simple aksharas and is not reduced by full fine-tuning, motivating targeted synthetic data — the subject of our companion work on Devanagari handwriting generation. Code, adapters, and the evaluation toolkit are available at <https://github.com/USER/DevGen>.

References

1. C3RL: IIIT-INDIC-HW-WORDS-Hindi (2024), <https://huggingface.co/datasets/c3rl/IIIT-INDIC-HW-WORDS-Hindi>
2. Chang, D., Li, Y.: DLoRA-TrOCR: Mixed text mode optical character recognition based on transformer (2024)
3. Gongidi, S., Jawahar, C.V.: IIIT-INDIC-HW-WORDS: A dataset for indic handwritten text recognition. In: Document Analysis and Recognition – ICDAR 2021. pp. 444–459 (2021). https://doi.org/10.1007/978-3-030-86337-1_30
4. Hu, E.J., Shen, Y., Wallis, P., Allen-Zhu, Z., Li, Y., Wang, S., Wang, L., Chen, W.: LoRA: Low-rank adaptation of large language models. In: International Conference on Learning Representations (2022), <https://openreview.net/forum?id=nZeVKeeFYf9>
5. Li, M., Lv, T., Chen, J., Cui, L., Lu, Y., Florencio, D., Zhang, C., Li, Z., Wei, F.: TrOCR: Transformer-based optical character recognition with pre-trained models. In: Proceedings of the AAAI Conference on Artificial Intelligence. vol. 37, pp. 13094–13102 (2023). <https://doi.org/10.1609/aaai.v37i11.26538>
6. Nikolaidou, K., Retsinas, G., Christlein, V., Seuret, M., Sfikas, G., Smith, E.B., Mokayed, H., Liwicki, M.: Wordstylist: Styled verbatim handwritten text generation with latent diffusion models. In: Document Analysis and Recognition – ICDAR 2023. pp. 384–401 (2023). https://doi.org/10.1007/978-3-031-41685-9_24
7. Nikolaidou, K., Retsinas, G., Sfikas, G., Liwicki, M.: Diffusionpen: Towards controlling the style of handwritten text generation (2024)
8. Paudel, A.: TrOCR Devanagari 2 (2024), <https://huggingface.co/paudelanil/trocr-devanagari-2>